

# Lab 5

Nicholas Storti

July 16, 2026

## 1 V1: Naive Implementation

### 1. Output:

```
Setting up the problem...0.006785 s
Input size = 1000000
Number of bins = 4096
Allocating device variables...0.160484 s
Copying data from host to device...0.000252 s
Launching kernel...0.022860 s
Copying data from device to host...0.000014 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.031759 s
Input size = 5000000
Number of bins = 8192
Allocating device variables...0.135438 s
Copying data from host to device...0.000949 s
Launching kernel...0.022470 s
Copying data from device to host...0.000021 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.062863 s
Input size = 10000000
Number of bins = 8192
Allocating device variables...0.139247 s
Copying data from host to device...0.001783 s
Launching kernel...0.022572 s
Copying data from device to host...0.000019 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.314396 s
Input size = 50000000
Number of bins = 8192
Allocating device variables...0.135520 s
Copying data from host to device...0.008402 s
Launching kernel...0.023510 s
Copying data from device to host...0.000021 s
Verifying results...TEST PASSED
```

```

Setting up the problem...0.628531 s
Input size = 100000000
Number of bins = 8192
Allocating device variables...0.139634 s
Copying data from host to device...0.016586 s
Launching kernel...0.024602 s
Copying data from device to host...0.000022 s
Verifying results...TEST PASSED

```

```

Setting up the problem...1.257417 s
Input size = 200000000
Number of bins = 8192
Allocating device variables...0.157768 s
Copying data from host to device...0.033136 s
Launching kernel...0.027095 s
Copying data from device to host...0.000022 s
Verifying results...TEST PASSED

```

## 2. Description of the Optimization

Version 1 is the naive implementation. Each item of the input array is assigned to its own thread. No memory optimizations such as shared memory are used.

## 3. Difficulties with Completing the Optimization Correctly

The primary difficulty of this implementation came from understanding how to use the atomicAdd operation and understanding how this function works.

## 4. Change in Execution Time after Optimization

This is the baseline execution time. The execution times of the optimizations will be compared to this one.

## 5. Explanation of Why Optimization Impacted Performance

No optimizations have been implemented yet, so this is the baseline that future optimizations will be compared to.

# 2 V2: Privatization with Global Memory

## 1. Output:

```

Setting up the problem...0.006800 s
Input size = 1000000
Number of bins = 4096
Allocating device variables...0.165829 s
Copying data from host to device...0.000253 s
Launching kernel...0.023071 s
Copying data from device to host...0.000017 s
Verifying results...TEST PASSED

```

```

Setting up the problem...0.031792 s
Input size = 5000000

```

```
Number of bins = 8192
Allocating device variables...0.140732 s
Copying data from host to device...0.000959 s
Launching kernel...0.024048 s
Copying data from device to host...0.000023 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.062918 s
Input size = 10000000
Number of bins = 8192
Allocating device variables...0.140293 s
Copying data from host to device...0.001793 s
Launching kernel...0.025311 s
Copying data from device to host...0.000024 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.314441 s
Input size = 50000000
Number of bins = 8192
Allocating device variables...0.135591 s
Copying data from host to device...0.008332 s
Launching kernel...0.035759 s
Copying data from device to host...0.000025 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.628881 s
Input size = 100000000
Number of bins = 8192
Allocating device variables...0.138034 s
Copying data from host to device...0.016671 s
Launching kernel...0.048770 s
Copying data from device to host...0.000028 s
Verifying results...TEST PASSED
```

```
Setting up the problem...1.257166 s
Input size = 200000000
Number of bins = 8192
Allocating device variables...0.137120 s
Copying data from host to device...0.033144 s
Launching kernel...0.074967 s
Copying data from device to host...0.000027 s
Verifying results...TEST PASSED
```

## 2. Description of the Optimization

This optimization implements privatization. Each block computes its own private copy of the histogram, then all of the private histograms are merged into the final product at the end. Each private copy for each block is stored in global memory. The intended benefit of this optimization is that it reduces the amount of contention for writing to the same memory locations. Multiple threads may be attempting to update the count at the same bin simultaneously. Having multiple private histograms

reduces the number of threads trying to update the same bin at the same time. The trade off is that the private copies need to be merged at the end of execution.

### 3. Difficulties with Completing the Optimization Correctly

The primary difficulty with this optimization is allocating sufficient global device memory for all of the private copies. The extra memory was added to the end of the bins array. Since each block needs its own private histogram with `num_bins` elements, `dim_grid.x * num_bins` total elements need to be allocated in the bins array.

### 4. Change in Execution Time after Optimization

Unfortunately, this optimization resulted in a severe increase in execution time over the naive implementation for all test cases. Particularly, the larger input arrays took a long time compared to the baseline.

### 5. Explanation of Why Optimization Impacted Performance

A possible reason for why this optimization hurt performance is that the private copies were stored in global memory. This becomes a problem at the end of execution during the merge operation. The private copies were already written to global memory, and now each private copy has to be read back from global memory, then written again to the final histogram in global memory. All of these global memory accesses result in high latency.

## 3 V3: Privatization with Shared Memory

### 1. Output:

```
Setting up the problem...0.006766 s
Input size = 1000000
Number of bins = 4096
Allocating device variables...0.164268 s
Copying data from host to device...0.000257 s
Launching kernel...0.023074 s
Copying data from device to host...0.000014 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.031783 s
Input size = 5000000
Number of bins = 8192
Allocating device variables...0.141354 s
Copying data from host to device...0.000922 s
Launching kernel...0.022719 s
Copying data from device to host...0.000020 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.062927 s
Input size = 10000000
Number of bins = 8192
Allocating device variables...0.136199 s
Copying data from host to device...0.001762 s
Launching kernel...0.022853 s
Copying data from device to host...0.000019 s
Verifying results...TEST PASSED
```

```

Setting up the problem...0.314305 s
Input size = 50000000
Number of bins = 8192
Allocating device variables...0.138140 s
Copying data from host to device...0.008316 s
Launching kernel...0.023733 s
Copying data from device to host...0.000021 s
Verifying results...TEST PASSED

```

```

Setting up the problem...0.628686 s
Input size = 100000000
Number of bins = 8192
Allocating device variables...0.134864 s
Copying data from host to device...0.016625 s
Launching kernel...0.024216 s
Copying data from device to host...0.000021 s
Verifying results...TEST PASSED

```

```

Setting up the problem...1.256467 s
Input size = 200000000
Number of bins = 8192
Allocating device variables...0.138617 s
Copying data from host to device...0.033084 s
Launching kernel...0.026325 s
Copying data from device to host...0.000023 s
Verifying results...TEST PASSED

```

## 2. Description of the Optimization

This optimization uses privatization like the previous version. However, instead of storing the private copy for each block in global memory, the private copies are stored in the shared memory for each block. Shared memory access is faster than global memory access, so this optimization should reduce some of the latency associated with global memory accesses while yielding the benefits of privatization.

## 3. Difficulties with Completing the Optimization Correctly

The primary difficulty in implementing this optimization correctly was in allocating the shared memory. Since the size of the shared memory needed for each private histogram is determined by the variable `num_bins`, the shared memory has to be dynamically allocated. This means that a third parameter indicating the size of the shared memory to be allocated must be passed inside the `jit::lll` portion of the kernel function call. The "extern" keyword also had to be used when declaring the shared memory array.

## 4. Change in Execution Time after Optimization

This optimization was still slightly slower than the naive implementation for smaller array sizes, but it did show increase in execution speed for the last two and largest array sizes. This optimization resulted in faster execution times for larger input arrays.

## 5. Explanation of Why Optimization Impacted Performance

A potential explanation for why the baseline still executed faster for smaller input array sizes is that the merging of the private histograms into the final histogram at the end of execution for the privatization optimization took extra time. However, for larger input array sizes, the extra time needed for merging was outweighed by the increase in speed gained by using private copies stored in shared memory.

## 4 V4: Thread Coarsening with Contiguous Partitioning

### 1. Output:

```
Setting up the problem...0.006800 s
Input size = 1000000
Number of bins = 4096
Allocating device variables...0.162040 s
Copying data from host to device...0.000258 s
Launching kernel...0.022832 s
Copying data from device to host...0.000015 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.031823 s
Input size = 5000000
Number of bins = 8192
Allocating device variables...0.141355 s
Copying data from host to device...0.000947 s
Launching kernel...0.022443 s
Copying data from device to host...0.000019 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.062853 s
Input size = 10000000
Number of bins = 8192
Allocating device variables...0.136029 s
Copying data from host to device...0.001802 s
Launching kernel...0.022469 s
Copying data from device to host...0.000019 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.314303 s
Input size = 50000000
Number of bins = 8192
Allocating device variables...0.140229 s
Copying data from host to device...0.008399 s
Launching kernel...0.023184 s
Copying data from device to host...0.000021 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.628638 s
Input size = 100000000
Number of bins = 8192
Allocating device variables...0.134638 s
Copying data from host to device...0.016684 s
Launching kernel...0.023743 s
Copying data from device to host...0.000021 s
Verifying results...TEST PASSED
```

```

Setting up the problem...1.257279 s
Input size = 200000000
Number of bins = 8192
Allocating device variables...0.154487 s
Copying data from host to device...0.033122 s
Launching kernel...0.025255 s
Copying data from device to host...0.000021 s
Verifying results...TEST PASSED

```

## 2. Description of the Optimization

This optimization implemented thread coarsening by having each thread process multiple input elements and update the histogram for each element instead of assigning one input element to each thread. In this implementation, each thread processed 16 input elements. The advantage of thread coarsening is that it reduces the total number of threads needed and thus reduces the total number of blocks. Reduced blocks reduces the number of merge operations needed at the end of execution to combine the private histogram copies. Since each block has its own private copy, fewer blocks means fewer copies and less overhead for merging them. This implementation used a contiguous partitioning strategy where the input elements processed by each thread are directly next to each other in the input array.

## 3. Difficulties with Completing the Optimization Correctly

The primary difficulty in implementing this optimization correctly was in the indexing of the input array. Each thread had to pull multiple elements from the input array, so the indexing of the input array had to be correct. The indexing had to insure that all the elements were processed and that there were no overlaps.

## 4. Change in Execution Time after Optimization

This optimization resulted in faster execution time for all test cases compared to the baseline and the optimization with privatization using shared memory. This optimization resulted in the fastest execution times for all test cases so far.

## 5. Explanation of Why Optimization Impacted Performance

A potential reason for why this optimization resulted in the best performance so far is that the thread coarsening reduced the overhead needed for merging the private histograms by reducing the number of blocks and number of private copies. This made the kernel perform better for smaller input array sizes compared to the version using just privatization and no coarsening. This also made the kernel perform better than the baseline for both small and large input array sizes.

# 5 V5: Thread Coarsening with Interleaved Partitioning

## 1. Output:

```

Setting up the problem...0.006829 s
Input size = 1000000
Number of bins = 4096
Allocating device variables...0.158040 s
Copying data from host to device...0.000257 s
Launching kernel...0.023077 s
Copying data from device to host...0.000013 s
Verifying results...TEST PASSED

```

```
Setting up the problem...0.031776 s
Input size = 5000000
Number of bins = 8192
Allocating device variables...0.135467 s
Copying data from host to device...0.000929 s
Launching kernel...0.022615 s
Copying data from device to host...0.000019 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.062896 s
Input size = 10000000
Number of bins = 8192
Allocating device variables...0.135336 s
Copying data from host to device...0.001783 s
Launching kernel...0.022600 s
Copying data from device to host...0.000020 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.314337 s
Input size = 50000000
Number of bins = 8192
Allocating device variables...0.138892 s
Copying data from host to device...0.008346 s
Launching kernel...0.023277 s
Copying data from device to host...0.000022 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.628708 s
Input size = 100000000
Number of bins = 8192
Allocating device variables...0.134302 s
Copying data from host to device...0.016534 s
Launching kernel...0.024469 s
Copying data from device to host...0.000021 s
Verifying results...TEST PASSED
```

```
Setting up the problem...1.257367 s
Input size = 200000000
Number of bins = 8192
Allocating device variables...0.136820 s
Copying data from host to device...0.033066 s
Launching kernel...0.026284 s
Copying data from device to host...0.000023 s
Verifying results...TEST PASSED
```

## 2. Description of the Optimization

This optimization also used thread coarsening. Instead of using a contiguous partitioning strategy to assign input elements to threads, this implementation used an interleaved partitioning strategy. Adjacent threads are assigned adjacent elements in the input array. The elements assigned to a single



thread are far apart in the input array, but elements in adjacent threads are adjacent in the input array. This means that when multiple threads go to read their elements from global memory, they will be reading elements that are close to each other in global memory, since they are reading elements that are close to each other in the input array. Groups of these elements can be read from global memory at the same time, reducing global memory access latency.

### 3. Difficulties with Completing the Optimization Correctly

As with the previous thread coarsening optimization, getting the indexing into the input array was challenging. Instead of having one thread read in adjacent elements, the indexing had to be such that adjacent threads read in adjacent elements.

### 4. Change in Execution Time after Optimization

The execution time was faster than the naive implementation for the last three and largest input array sizes. It was slower than the baseline for the smaller array sizes.

### 5. Explanation of Why Optimization Impacted Performance

Due to the thread coarsening and shared memory privatization, this optimization resulted in faster execution times compared to the baseline for larger input array sizes.

## 6 V6: Aggregation

### 1. Output:

```
Setting up the problem...0.006691 s
Input size = 1000000
Number of bins = 4096
Allocating device variables...0.165505 s
Copying data from host to device...0.000254 s
Launching kernel...0.022837 s
Copying data from device to host...0.000014 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.031774 s
Input size = 5000000
Number of bins = 8192
Allocating device variables...0.141098 s
Copying data from host to device...0.000959 s
Launching kernel...0.022471 s
Copying data from device to host...0.000020 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.062919 s
Input size = 10000000
Number of bins = 8192
Allocating device variables...0.134731 s
Copying data from host to device...0.001792 s
Launching kernel...0.022550 s
Copying data from device to host...0.000020 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.314385 s
```

```
Input size = 50000000
Number of bins = 8192
Allocating device variables...0.145110 s
Copying data from host to device...0.008384 s
Launching kernel...0.023325 s
Copying data from device to host...0.000021 s
Verifying results...TEST PASSED
```

```
Setting up the problem...0.629265 s
Input size = 100000000
Number of bins = 8192
Allocating device variables...0.140080 s
Copying data from host to device...0.016712 s
Launching kernel...0.024171 s
Copying data from device to host...0.000022 s
Verifying results...TEST PASSED
```

```
Setting up the problem...1.257152 s
Input size = 200000000
Number of bins = 8192
Allocating device variables...0.149316 s
Copying data from host to device...0.033144 s
Launching kernel...0.026316 s
Copying data from device to host...0.000031 s
Verifying results...TEST PASSED
```

## 2. Description of the Optimization

This optimization implements aggregation. For this optimization, consecutive updates are aggregated into a single update. If consecutive values in the input array are the same, a counter accumulates the count of this repeated value until a new value is reached in the input array. The histogram is only updated with this accumulated count when a new value is reached in the input array. The intention of this optimization is to reduce the number of memory accesses needed to update the histogram.

## 3. Difficulties with Completing the Optimization Correctly

The difficulty in implementing this optimization came from the control logic for updating the accumulator and checking if a new value was found in the input array.

## 4. Change in Execution Time after Optimization

This optimization resulted in slightly faster execution times over the naive implementation for some test cases and slightly slower execution times for other test cases.

## 5. Explanation of Why Optimization Impacted Performance

A possible explanation for the inconsistent change in execution time over the naive implementation is that the input arrays are determined randomly. Aggregation performs better when there are large numbers of consecutive elements. Since the arrays are random, the number of consecutive elements is also random. For some test cases, there were a lot of consecutive elements, so aggregation performed better. For others, the opposite occurred. Appropriate use of aggregation would require knowledge of the nature of the input data set. It would only be a good idea to use aggregation if it is known before hand that the input array will contain large sections of consecutive elements.